1. このマニュアルの使い方・最初のプログラム

v0.25.0 対応版 2026年11月1日改訂

このマニュアルは、開発した順ではなく、プログラム画面の左側にあるブロックグループの順で整理しています。「使うブロックのグループを開く → 同じ名前の章を読む」という探し方ができます。各章では設定・基本命令・使い方を説明します。センサ関連は第7章に集約しています。

最初に第2章で保存・接続・転送を確認し、次の作例を試してください。ブロックの数値や条件には、形が合う変数・引数・演算・センサ値を入れられます。

最初の作例

スタート → 動きに使うモーターを左B・右Cにする → タイヤの直径を56mmにする → 速さ20%で20cm前に進む。

サンプル「共通設定で前進」で読み込めます。実物の左右ポートとタイヤ直径に合わせて変更してください。スピン・ピボットではトレッドの設定も必要です。モーターを使わず接続を試す場合は「音だけ」のサンプルを使います。

グループ別の目次

章	内容
1	このマニュアルの使い方・最初のプログラム
2	画面・保存・接続と転送
3	スタート
4	イベント
5	モーター
6	動き
7	センサー
8	音
9	制御
10	演算
11	画面
12	ランプ
13	データ
14	記録
15	変数
16	マイブロック
17	引数
18	授業の作例・サンプルと実機テスト
19	エラーと制限
20	EV3 Classroomとの機能比較と開発の目的

対象と確認範囲

標準のLEGO EV3ファームウェアで動作する試作アプリです。本体へ保存した実行プログラムはPCを切り離して実行できます。WindowsでのBluetooth転送・基本動作は所有者による実機確認があります。追加機能・USB・Chromebookの全組み合わせは実機検証を継続中で、自動テストの合格を実機確認済みとは扱いません。

ブロック仕様はv0.25.0のソースに基づきます。学年別表示・「何マス進む」表示は未実装です。旧作品に左右設定を自動補完しません。編集エリア下のアドバイスを確認し、設定と配線を見直してください。

2. 画面・保存・接続と転送

画面と保存

プロジェクト名はEV3のフォルダ名、プログラム名はファイル名です。編集用ファイルとEV3実行用ファイルを使い分けます。

操作	役割・入力	使い方と注意
プロジェクト名	半角英数字、ハイフン、アンダースコア。1～31文字。	EV3上の保存先フォルダ。例：Class3
プログラム名	同じく1～31文字。拡張子は入力しない。	例：LineTrace → 保存はLineTrace.ev3lab、転送はLineTrace.rbf。
保存	編集内容をファイルに保存。	プログラム名.ev3lab。未接続の試作ブロックも含む。
開く	本アプリの作品ファイルを読み込む。	現在の作品を置き換える。必要なら先に保存する。
＋ 新しいプログラム	編集画面を新しくする。	マイブロックを再利用するときは先に書き出す。
その他 → マイブロックを書き出す	定義と参照する変数・引数を保存。	my-blocks.ev3blocksとして保存。
その他 → マイブロックを読み込む	現在の作品へ定義を追加。	同名の定義は上書きしない。変更してから読み込む。
サンプル	動作を確認する作品へ置き換える。	音・モーター・移動・センサ・加減速などを用意。
元に戻す／やり直す	直前の編集を取り消す／戻す。	ファイルを開いた直後は編集履歴が新しくなる。

ブロックの取り出しと複製

カテゴリを選ぶとブロック一覧が開きます。ブロックを取り出しても一覧は開いたままです。別のカテゴリを選ぶと切り替わり、編集エリアの白い余白をクリックすると閉じます。

ブロックを右クリックして「複製」を選ぶと、そのブロックから下につながる命令・入力値・入れ子の処理をまとめて複製します。上につながるブロックは複製しません。元のブロックは残り、「元に戻す」で複製を取り消せます。

自動保存とバックアップ

自動保存は、現在の作品を同じブラウザ内に保存します。端末の自動シャットダウン後も同じサイトとブラウザで復元を試みますが、閲覧データ消去・別端末・別のサイトURLには引き継がれません。授業の区切りで「保存」も行ってください。

GitHubへのバックアップはアプリのソースコードが対象です。児童がブラウザで作った作品やマイブロックが、自動でGitHubへ保存されるわけではありません。

ファイルの違い

.ev3lab＝編集用作品、.ev3blocks＝マイブロック集、.rbf＝EV3実行用。本アプリの旧.ev3／.jsonも読み込めますが、LEGO純正ソフトの.ev3形式とは互換性がありません。

接続と転送

右上の「接続・転送」で設定を開きます。転送だけのボタンと、転送後に起動するボタンは動作が異なります。

操作	動作	確認すること
Bluetoothで接続	OSでEV3をペアリングし、Web Serialの接続先を選ぶ。	Windowsでは名前ではなくCOM番号になる場合がある。対応する発信ポートを選ぶ。

操作	動作	確認すること
USBで接続	接続方法をUSBにして、WebHIDのデバイスを選ぶ。	データ通信できるケーブルをEV3のPC端子へ接続。標準起動のまま使う。
EV3に転送・照合	保存後、読み戻した全バイトを比較する。	起動はしない。照合後に短い2音を送る。
転送して実行	保存・照合後に今回のプログラムを起動する。	直後にモーターが動く場合がある。
実行を停止	本体のユーザープログラムと全モーターへ停止命令を送る。	接続中のみ有効。本体の戻るボタンでも停止できる。
切断	ブラウザとの通信を終了する。	切断は、実行中プログラムを止める操作ではない。
実行ファイルを保存	.rbfをPCにダウンロードする。	この操作だけではEV3への転送にならない。
ポートの接続機器と値	入力1～4のセンサ名・モード・値、出力A～DのL/Mモーター名・角度を表示。	「値を読む」か自動更新を使う。未接続と未対応機器は区別して表示。

本体だけで動かす手順

1. 有効なプロジェクト名とプログラム名を入力し、生成エラーがないことを確認する。
2. 「EV3に転送・照合」を押し、照合完了を確認する。
3. 切断後、EV3のファイル画面で指定したフォルダとプログラムを選び、実行する。

授業中に接続できないとき

他のEV3アプリや接続済みの別タブを閉じます。Bluetoothが不安定な場合は切断してからUSBへ切り替えます。学校の管理設定でWeb Serial／WebHIDが禁止されている場合は端末管理者の確認が必要です。

同じ保存先と同じプログラム名への転送は上書きです。以前のプログラムを残す場合は名前を変更してください。実行中のプログラムは停止してから転送します。

3. スタート

「スタート」を実行すると、その下につながった処理を上から順に行います。離して置いた試作ブロックや呼び出していない定義は、作品には保存しますがEV3へは転送しません。複数のスタートを置くと並行処理になります。

複数のスタートと走行・計測の並行処理

スタートを2個以上置き、それぞれに命令をつなぎます。モーターの角度・時間指定を待つ処理とは別に、センサ値の取得やCSV記録を続けられます。処理はEV3本体で動き、ブラウザの接続は不要です。

2秒の走行と距離の記録

スタート1：走行担当	スタート2：計測担当
B+Cを速さ20%で2秒動かす	CSVログParallelを開始する
計測完了＝1まで待つ	20回繰り返す［超音波2の距離を記録 → 0.1秒待つ］
プログラムを終了する	CSV記録を終了し、計測完了を1にする

通常変数の開始値は0です。開始順は保証しないため、別のスタートで初期化することに依存しないようにします。タイヤ直径・トレッドを使う命令は、その走行処理内で設定を先に実行します。

共有するもの・独立するもの

データ	扱い
通常変数・文字列変数・数値リスト	全処理で共有。同じ名前を使うと同じ値を参照する。
マイブロックの引数・作業領域	呼び出す処理ごとに独立。同じ定義を別のスタートから使える。
タイヤ直径・トレッド・加減速・停止方法・タイマー	共有。実行順を決めて設定する。
モーターポート	同じポートへ後から出した命令が動作を変更する。担当処理を決める。
センサモード	同じポートを別の処理から使うときはモードをそろえる。

スタートだけの構成では、すべてのスタートが終わると全モーターを停止します。一方だけが終わっても、もう一方の走行は継続します。イベント・受信処理がある場合は監視を継続します。「プログラムを終了する」で全処理を終了できます。

4. イベント

センサの変化によるイベント

「イベント」グループには、条件成立と数値変化の2種類の開始ブロックがあります。条件欄にはセンサ・比較・論理ブロックを入れます。上にほかのブロックをつなぎません。

ブロック	起動条件	注意
条件が成り立ったとき	偽から真へ変わったとき。開始時に真でも1回起動。	真のままでは繰り返さない。
数値の値が変わったとき	開始時の値を基準に、その後の変化で起動。	小さな揺れでも変化になる。しきい値の判断には条件イベントを使う。

タッチで走行を止める

スタート：B+Cを速さ20%で回し続ける → ずっと［0.1秒待つ］。

イベント：タッチセンサ1が押されている条件が成り立ったとき → プログラムを終了する。

スタート側をすぐ終わらせてもイベント監視は続きますが、連続回転の意図が分かるよう待つ処理を残す例にしています。

監視では読み取り・処理の後に約20ミリ秒待ちます。厳密な20ミリ秒周期ではありません。同じイベントの実行中は重ねて起動せず、その間の短い変化を取り逃す場合があります。実行後、偽を確認してから次の成立で再起動します。

同じカラーセンサの「色」と「反射光」を別処理から同時に読む構成は、モードが異なるため生成エラーになります。赤外線の近さ・ビーコン・リモコンも同様です。

メッセージで処理を連携する

ブロック	動作
名前を送る	同名の受信処理を起動し、すぐ次へ進む。
名前を送って待つ	同名の受信処理がすべて終わるまで待つ。受信側がなければすぐ次へ。
名前を受け取ったとき	同名メッセージで起動する開始ブロック。複数置ける。

メッセージ名は1〜40文字、32種類までです。大文字・小文字も含め一致させます。これは1台のEV3のプログラム内の連携であり、別のEV3との無線メッセージ通信ではありません。

計測の完了を待つ

スタート：メッセージ「測定」を送って待つ → 本体ランプを赤にする。

受信：測定を受け取ったとき → CSVログMessageを開始 → 10回［超音波2の距離を記録 → 0.1秒待つ］ → CSV記録終了。

受信処理を重ねて起動しません。処理中に受けた追加メッセージはまとめて保持し、終了後に最新分をもう1回実行します。全メッセージを1件ずつ順番に処理するキューではありません。

自分自身の受信完了を待ったり、2つの受信処理が互いの完了を待ったりする構成は生成エラーになります。受信側に「ずっと」があると完了しないため、送信側の「送って待つ」も進みません。

CSV・リスト・通常変数などの変更は内部で順番に処理します。ただし、別の処理でCSVを開始し直すと同じ共有の記録先が変更されます。CSVの開始と終了は担当処理を1つにし、記録完了を待って閉じてください。

5. モーター

単独モーターと停止

「モーター」には、LモーターとMモーターそれぞれに「1個」と「2個」のグループがあります。このページは1個の制御です。

ブロック	入力と動作	使用例
Lモーター1個を回し続ける	A〜D、速さ−100〜100%。回転開始後すぐ次へ進む。0は停止。	Bを20%で回し続ける → 1秒待つ → Bを停止。
Mモーター1個を回し続ける	同じ入力と動作。負の速さは逆回転。	Aを−20%で回し続ける。
Lモーター1個を指定角度回す	速さ−100〜100%で0以外。角度1〜36000度。完了を待って次へ進む。	Bを20%で360度回す。
Mモーター1個を指定角度回す	同じ入力範囲。角度はモーター軸の回転量。	Aを−30%で90度回す。
Lモーター1個を指定時間動かす	速さは0以外。時間は0より大きく60秒以下。小数可。完了を待つ。	Bを30%で1.5秒動かす。
Mモーター1個を指定時間動かす	同じ入力範囲。時間の単位は秒。	Aを20%で0.5秒動かす。
モーターを選んだ方法で停止	A〜D、ポートの組、すべてから選択。ブレーキ・惰性・保持を直接選ぶ。詳細は第5章。	Bを停止しても、Cへの連続回転命令は続く。
すべてのモーターを停止	A〜Dへ従来の標準FWの停止命令を送り、次へ進む。停止方法を選びたい場合は上の停止ブロックで「すべて」を選ぶ。	停止 → 音を鳴らす、という並びも可能。

L・M／1個・2個それぞれに回転数指定もあります。1回転＝360度、0.5回転＝180度です。速さを負にして逆回転します。換算後の角度は1〜36000度の範囲が必要です。

回し続ける命令の後には待つ処理を置く

連続回転命令だけでプログラムの末尾へ到達すると、終了時に全モーターが停止します。「回し続ける → 待つ」や「回し続ける → 条件まで待つ」と組み合わせます。

角度と速さの見方

360度はモーター軸1回転です。歯車を介した車輪やアームの角度とは一致しない場合があります。逆方向へ回すときは、角度を正の値にしたまま速さを負にします。速さ%は角速度そのものではなく、実際の速さは種類や負荷により変わります。

速さ%と角度は整数として命令に渡されます。演算結果に小数がある場合は、命令へ渡す段階で小数部分を0方向へ切り捨てます。

2個のモーター

2個のグループではA+B、A+C、A+D、B+C、B+D、C+Dを選べます。種類をそろえたモーターで確認してください。

ブロック	入力と動作	使用例
Lモーター2個を回し続ける	選んだ2個へ同じ速さを指定して始動。すぐ次へ進む。	B+Cを20% → 距離条件まで待つ → 全停止。
Mモーター2個を回し続ける	同様。連続回転では角度や停止時刻の補正を行わない。	A+Dを−20%で回し続ける。
Lモーター2個を指定角度回す	速さは0以外、角度1〜36000度。設定された方式で2個の完了を待つ。	B+Cを30%で360度回す。
Mモーター2個を指定角度回す	同様。制御方式は先生用の設定で選ぶ。	A+Dを20%で180度回す。

ブロック	入力と動作	使用例
Lモーター2個を指定時間動かす	時間は0より大きく60秒以下。2個を同時に指定時間だけ動かす。	B+Cを30%で2秒動かす。
Mモーター2個を指定時間動かす	同様。時間指定は角度をそろえる命令ではない。	A+Dを−20%で0.5秒動かす。

先生用の設定と計測にある角度制御の方式

方式	何を優先するか	注意
各モーターの指定角度を優先	各軸が指定角度に達するまで動く。	止まる時刻は異なる場合がある。
停止タイミングを補正 試作	進行角度を比較し、先行側の速度を調整する。	角度を優先する試作補正。停止時刻の完全一致は保証しない。
EV3同期制御 検証用	EV3の同期命令で組を制御する。	実機で角度や終了時刻を検証して使う。

設定を変えたときは転送し直します。加減速を有効にした有限移動では加減速用の制御が使われ、上の方式と同じ動きにはならない場合があります。

左右で別々の速度を使いたい場合は、第6章の「左右の速さで回し続ける」を使います。片側だけ停止したい場合は、第5章のポート別停止を使います。

動作後の停止方法・速度・パワー

停止ブロックで直接選ぶ

「モーター A を ブレーキ で停止」は、ここで選んだ方法で直ちに停止します。初期値はブレーキです。ポートはA〜D、2個の組、すべてから選べます。

方法	このアプリでの動作
ブレーキ	標準FWの制動を100ミリ秒行ってから力を抜く試作方式。実際の停止には負荷による差がある。
惰性	すぐに力を抜く。慣性で余分に回る場合がある。
保持	標準FWの位置保持を使い、停止位置を保とうとする。プログラム実行中のみ有効。

保持を確認するときは、後ろに「1秒待つ」などを置きます。プログラム終了・電源断後の保持は保証しません。別の処理が同じポートへ命令すると保持状態も変わります。

動作後の停止方法を設定する

「モーターの動作後の停止方法を…にする」は、後ろの角度・回転数・時間・距離・旋回が終わったときの設定です。設定ブロックだけでは停止や回転を行いません。直接停止ブロックは、そのブロックで選んだ方法を優先します。

旧作品の「設定どおり」は「『動作後の停止方法』の設定」と表示します。以前の動作は維持され、設定がない旧作品は従来の標準FWの制動を使います。標準FWの制動には位置保持の働きもあるため、上の短時間ブレーキと同一ではありません。

同期走行・ステアリングの2ポートは同じ停止方法に設定します。異なる設定のまま実行すると、同期走行を開始せず、エラー音で全体を停止します。

速度指定とパワー指定

ブロック	意味
速さ%で回し続ける	指定の速度を保つよう本体が出力を調整。
速さを%にする	動作中の速度を変更。停止中のモーターをこの命令だけで始動しない。角度制御の目標は維持。
パワー%で回し続ける	出力率を固定。負荷で速度が変わる。同じ20%でも速度指定20%とは異なる。
実際の速さ（センサー）	A〜Dの実測回転速度を−100〜100%で返す。逆回転は負。

L・M共通で速度・パワーは−100〜100%。小数は命令へ渡す段階で0方向へ切り捨てます。パワー0は出力0であり、位置保持ではありません。加減速・同期制御中に同じポートの速度・パワーを別処理から変更すると制御が干渉します。

台形とS字の加減速

加減速のブロックは、それ以降に実行する有限移動の設定です。このブロックだけではモーターは回りません。

ブロック	入力	意味
加減速を方式にする 加速 秒 減速 秒	方式＝台形／S字／なし。加速と減速は各0〜10秒。	後続のL/M角度・時間指定、前後移動、スピン、ピボットに適用。

方式	速度の変わり方	使いどころ
台形	速さの指令を直線的に増やし、減速時は直線的に下げる。	動き始めと停止を急に変えたくないとき。
S字	速度の変化を滑らかにする曲線を使う。	台形と動きの違いを比べるとき。
なし	後続の有限移動に加減速設定を適用しない。	通常の角度・時間指定へ戻すとき。

同じ条件で台形とS字を比較する

1. タイヤの直径を56mmにする。
2. 加減速を台形にする。加速1秒、減速1秒。
3. 動きに使うモーターを左B・右Cに設定し、速さ30%で60cm前に進める。
4. 1秒待つ。
5. 加減速をS字にする。加速1秒、減速1秒。
6. 同じく速さ30%で60cm前に進める。

設定が適用されない命令

「回し続ける」「左右の速さで回し続ける」「停止」「パワー指定」「ステアリング」「左右別速度の角度・時間指定」は対象外です。設定はプログラム内で保持されます。マイブロックで変更した場合も、それ以降の命令に影響します。

短い移動と試作上の制限

短い距離や時間では指定した最高速度に到達しない場合があります。角度指定は停止位置を優先するため、設定秒数どおりの減速にならない場合があります。本体で約20msごとに速度指令を更新する試作方式です。

この版のS字は滑らかな速度曲線を作る方式です。一定ジャークを7区間で切り替える方式ではありません。実際の停止位置は負荷や床の滑りも含めて確認してください。

6. 動き

共通の左右モーター設定

v0.23.0では「動き」の各ブロックにあったポート欄を取り除き、左右の配線を1つの設定ブロックにまとめました。「モーター」の単独・2個回転、速度変更、パワー、停止方法、停止命令には引き続きポート欄があります。アームなど独立した機構に使えます。

最初に使うブロック

「動きに使うモーターを 左 B 右 C にする」をスタートの直後につなぎます。左右はA〜Dから別々に選び、同じポートはエラーです。B・CとC・Bは別の設定です。左右はロボットが前進するときの向きを基準にし、実際の配線を確認します。初期表示はB・Cですが、プログラムに暗黙の既定設定はありません。

このブロックだけではモーターを回しません。編集エリアに離して置くだけでも設定は有効になりません。実行された時点から、その後の動き・マイブロック・並行処理・イベントで共有します。タイヤ直径、トレッド、速度や距離の範囲、ブロックの色は従来どおりです。距離・スピン・ピボットはLモーター直結、ステアリング・左右別速度は同じ種類のL同士またはM同士を使います。

動きの並び順	ブロックの入力
左右モーター設定 → タイヤ直径 → トレッド	配線は設定だけで選ぶ。寸法は従来の単位。
ステアリング連続 → 角度 → 時間	曲がり方・速さ・必要な角度または秒数。
左右別速度の角度 → 時間 → 連続	左右の速さと必要な角度または秒数。
右／左スピン → 右／左ピボット	速さとロボットの旋回角度。
前進 → 後退	速さと距離cm。

基本例とマイブロック

基本例：スタート → 動きに使うモーターを左B・右Cにする → タイヤの直径を56mmにする → 速さ20%で20cm前に進む。

サンプル「共通設定で前進」（MovementBasic.ev3lab）がこの構成です。「共通設定とマイブロック」（MovementMyBlock.ev3lab）では呼び出し側で左右と直径を設定し、定義「前進」の中にはポート欄のない前進ブロックを置きます。距離の引数を変えて再利用できます。寸法と配線の設定を「定義」などのマイブロックにまとめてから呼び出しても使えます。

途中で組を変える

サンプル「途中で左右モーター変更」（MovementSwitch.ev3lab）：左B・右C → 直径56mm → 20%で20cm前進 → 1秒待つ → 左A・右Dへ変更 → 20%で20cm後退。4個のモーターを使う例なので、試験する実機の配線に必ず合わせて変更します。

変更ブロックを実行すると、以前の左右モーターにブレーキ停止命令を送り、その後で新しい左右設定に切り替えます。停止までの機械的な時間を保証するものではありません。同じ左右設定を繰り返しても、不要な停止や再登録は行いません。B・CからC・Bへの変更も左右の意味が変わるため旧組を停止します。マイブロックから戻っても設定は自動で元に戻りません。

新しくプログラムを実行するたびに未設定へ戻ります。本体の電源を入れ直した後も同じです。保存された実行プログラムが先頭から設定を実行するため、以前の実行の配線状態に依存しません。

設定が不足している場合

「動きに使うモーターを設定してください」と生成・転送前に表示します。未接続の設定や、呼び出していない定義の設定は数えません。単一スタートでは、動きより後の設定、片側の条件分岐だけで行う設定、0回かもしれない繰り返しだけの設定、早期復帰して設定を通らないマイブロックなども検出します。両方の分岐で設定する構成や、設定済みの呼び出し元から動きの定義を使う構成は扱えます。

並行処理・イベントの順序は静的には保証できません。他の処理に設定があるだけでは、先に設定済みとは扱いません。EV3本体でも各動きの開始前に必ず共有設定を確認し、未設定なら全モーター停止・×とエラー番号23の画面表示・エラー音・プログラム終了を行います。番号23は「動きの左右モーターが未設定」の意味です。設定が済んでからメッセージを送って動作側を始めるなど、順序を明確にしてください。

同じモーターに複数の処理から同時に命令したり、動作中に別処理から左右設定を変更したりしないでください。配線設定は共有ですが、走行完了まで他の計測処理を止める仕組みではありません。センサの入力1〜4とモーターの出力A〜Dは別の物理端子です。同一の端子を異なる機器・モードとして利用する従来の検査は維持されます。実際の配線違いは未設定とは別の問題として確認します。

旧作品を読み込む場合

以前の動きブロックのポート欄は新しい画面に表示しません。旧作品からB+Cを黙って選ぶことはありません。「動きに使うモーター」を追加し、左右と実機配線を確認してから生成・転送してください。旧データにポート欄が残っていても、共通設定を上書きする値には使いません。編集データを保存し直すと新しい形になります。

タイヤ直径・トレッドと距離移動

長さからモーター角度へ換算するため、距離で進む前にタイヤの直径を設定します。

ブロック	入力と範囲	動作と使用例
タイヤの直径を mm にする	10〜200mm。0.1mm刻み。	例：56mm。タイヤの外径を測る。後続の移動・旋回に使う。
トレッドを mm にする	10〜1000mm。0.1mm刻み。	例：112mm。左右タイヤの中心どうしの間隔。旋回に必要。
速さ%で cm 前に進む	共通設定の左右2個。速さ1〜100%、長さ1〜1000cm。	例：事前に左B・右Cを設定し、30%、20cm。移動完了を待って次へ進む。
速さ%で cm 後ろに下がる	同じ入力範囲。長さと速さは正の数で指定。	例：同じ共通設定で20%、10cm。命令が後退方向を指定する。

基本の並べ方

スタート → 動きに使うモーターを左B・右Cにする → タイヤの直径を56mmにする → 速さ20%で20cm前に進む → 1秒待つ → 速さ20%で10cm後ろに下がる。

直径とトレッドをまとめる

マイブロック「初期設定」の中に、直径とトレッドの設定を入れられます。スタートから先に呼び出してください。定義を置いただけでは実行されません。設定前に移動すると生成エラーになります。

距離の考え方

理論上は「モーター回転角度＝進む距離÷タイヤの円周×360」です。アプリ内で必要な単位換算と整数角度への丸めを行います。直径56mmで20cm進む場合、モーターの回転量は約409度です。

タイヤはLモーターに直結し、両方の正回転で前へ進む取り付けを前提にしています。歯車比の設定には未対応です。

距離cmはこの版では整数として扱います。車輪の空転、タイヤの変形、床の摩擦などで実際の距離は変わります。数値だけで位置の正確さを保証するものではありません。

ステアリングの連続・角度・時間

「動き」グループに、連続・角度・時間の3種類を追加しました。先に共通設定ブロックで左右に異なるポートを選び、各走行ブロックではステアリング値で曲がり方を指定します。L/M共通です。

命令	入力	次へ進むとき
ステアリングで回し続ける	共通設定の左右、ステアリング−100〜100、速さ−100〜100%。	開始直後。停止命令を続けて使う。
ステアリングで外側を 度回す	同じ入力。角度1〜36000度。速さ0は不可。	指定した走行が終わった後。

命令	入力	次へ進むとき
ステアリングで 秒走る	同じ入力。時間は0より大きく60秒以下。速さ0は不可。	指定時間の走行が終わった後。

ステアリング値	前進時の動き	車輪の速さの目安
−100	左スピン	左は逆、右は正。
−50	左ピボット	左は停止、右は正。
−25	左カーブ	左が遅く、右が速い。
0	直進	左右とも同じ速さ。
25	右カーブ	左が速く、右が遅い。
50	右ピボット	左は正、右は停止。
100	右スピン	左は正、右は逆。

角度は外側モーターの軸の回転量です。ロボット自体の向きの角度ではありません。ロボットを指定角度だけ向けたい場合は、直径とトレッドを設定したスピン・ピボットを使います。

両輪の正回転が前進となる取り付けを前提にします。速さを負にすると逆回転です。加減速設定はこのステアリング命令には適用しません。曲がる量はタイヤ、床、負荷にも左右されます。

左右のカーブを比較する

1. 動きに使うモーターを左B・右Cに設定し、ステアリング25、速さ20%で2秒走る。
2. 1秒待つ。
3. ステアリング−25、速さ20%で2秒走る。

左右別速度の角度・時間

左右別速度で角度・時間を指定する

共通設定を左B・右Cにし、左30%・右15%で「速い側を360度回す」と、左は360度、右は約180度です。両方を同じ角度まで回す命令ではありません。左右を交換すると基準の側も変わります。

同じ共通設定で左30%・右−30%で「0.5秒走る」と、左右を逆向きに同時に動かします。速さは各−100〜100%で両方0は禁止。角度は1〜36000度、時間は0.001〜60秒です。比率は標準FWで約1%刻みになり、台形・S字の加減速設定は適用しません。

左右別速度の連続走行

左右の速さを別々に設定すると、直進・後退・カーブ・その場での旋回を作れます。

ブロック	入力	動作
左の速さ% 右の速さ%で回し続ける	左右は共通設定。各速さ−100〜100%。	設定後すぐ次へ進む。0の側はブレーキ停止。次の命令や停止、プログラム終了まで回る。

左の速さ	右の速さ	動きの例
30	30	前進。
−30	−30	後退。
30	15	右へ曲がるカーブ。
15	30	左へ曲がるカーブ。
30	−30	右へその場で旋回。
−30	30	左へその場で旋回。

左の速さ	右の速さ	動きの例
30	0	右輪を止めた右ピボットの動き。
0	30	左輪を止めた左ピボットの動き。

これらは両輪の正回転が前進となる取り付けの例です。指定値は基準速度に対する比率ではなく、左右それぞれへの速度指令です。負荷によって実際の速度比は変わります。L/M共通で使えますが、加減速と角度同期の補正は行いません。

スピンターンとピボットターン

先に「動きに使うモーター」で左右を設定します。旋回ブロックにはポート欄がありません。ここで指定する角度はロボットの旋回角度です。モーターの回転角度とは区別します。直径とトレッドを先に設定してください。

ブロック	動く車輪	入力
右に度スピンターン	左輪は正転、右輪は逆転。	共通設定の左右を使用。速さ1〜100%、角度1〜3600度。
左に度スピンターン	左輪は逆転、右輪は正転。	同じ入力範囲。
右に度ピボットターン	右輪をブレーキ停止し、左輪だけ正転。	同じ入力範囲。
左に度ピボットターン	左輪をブレーキ停止し、右輪だけ正転。	同じ入力範囲。

モーターの回転量の換算

スピン：各モーターの回転量＝トレッド÷タイヤ直径×ロボットの旋回角度。

ピボット：動くモーターの回転量＝2×トレッド÷タイヤ直径×ロボットの旋回角度。

設定と目標	スピン	ピボット
直径56mm、トレッド112mm、右へ90度	左輪＋180度、右輪−180度。	左輪＋360度、右輪は停止。
同じ寸法で左へ90度	左輪−180度、右輪＋180度。	左輪は停止、右輪＋360度。

直進後に右へ向きを変える

1. タイヤの直径を56mm、トレッドを112mmにする。
2. 動きに使うモーターを左B・右Cにし、速さ20%で20cm前に進む。
3. 速さ20%で右に90度スピンターンする（共通設定を使用）。
4. 速さ20%で20cm前に進む。

ポートと取り付けを確認する

左右はロボットが前へ進む向きを基準にします。Bが必ず左とは限りません。左右の取り付けが逆の場合はポートを入れ替えて指定してください。上の符号は正転で前進する構成を前提にしています。

換算後のモーター角度は1〜36000度の範囲が必要です。床の滑りで向きはずれるため、同じ場所で何回か試して確認します。ジャイロによる向きの補正は未対応です。

7. センサー

バッテリー電圧・電流

v0.22.0で追加した機能です。実機検証は継続中です。

バッテリーの2つのブロック

「センサー」グループの「EV3 バッテリー」にあります。ポートの選択は不要です。

ブロック	返す数値
EV3本体のバッテリー電圧（V）	バッテリー全体の電圧。単位はV、小数あり。残量の%ではありません。
EV3本体のバッテリー電流（A）	本体・接続機器を含むバッテリー全体の消費電流。単位はA、小数あり。各モーターの電流ではありません。

電流をmAで扱う場合は1000を掛けます。電圧・電流は、変数・比較・演算・CSV記録・本体画面の数値表示に使えます。例えば「電圧×電流」で消費電力の目安をW単位で求められますが、2つの値は順番に取得するため、厳密な同時測定ではありません。本体の測定値であり、校正された計測器としての精度を保証しません。

バッテリーの測定命令

LEGO EV3 Firmware Developer Kit (https://assets.education.lego.com/v3/assets/blt293eea581807678a/blt469be1e11ad37696/5f880384f71916144453a49f/lego-mindstorms-ev3-firmware-developer-kit.pdf?locale=en-us) の UI_READ GET_VBATT（V）・GET_IBATT（A）を使います。ファームウェアは書き換えません。

タイマーと旧作品用の本体ボタン

タイマーはプログラム開始またはリセットからの経過時間を読みます。本体ボタンは現在、新規の一覧から一時的に非表示です。以前保存した作品にあるボタンブロックは復元できます。以下のボタン説明は互換用です。

ブロック	入力・値	使用例
本体のボタンが状態	上・下・左・右・中央・いずれか。押されている／離されているを判定。	右ボタンが押されている、まで待つ。
本体のボタンの値 0／1	押下1、解放0。いずれかは5個のうち1個以上の押下。	押下値に本体の中央ボタンの値をセット。
タイマー 秒	開始またはリセットからの経過秒数。小数あり。	タイマーが3以上、まで待つ。
タイマーをリセット	共通のタイマーを0秒へ戻す。	計測区間の直前に入れる。

右ボタンを押すたびに1回だけ音を鳴らす（旧作品用）

1. ずっと、の中に次の3つを入れる。
2. 右ボタンが押されている、まで待つ。
3. 音を鳴らす。
4. 右ボタンが離されている、まで待つ。

押している間は条件が成り立ち続けます。押した瞬間を記憶するイベントではありません。中央ボタンで本体から起動した直後は、まだ中央が押されている場合があります。必要なら最初に離されるまで待ちます。戻るボタンは停止操作用で、判定の選択肢には含めません。

3秒間走る区間を作る

1. タイマーをリセットする。
2. B+Cを20%で回し続ける。
3. タイマーが3以上、まで待つ。

4. すべてのモーターを停止する。

タイマーはプログラム全体で1個を共有します。マイブロック内でリセットしても共通です。リセットは待ち時間やモーター制御の時計を変更しません。実際の停止は命令の処理時間や負荷にも依存します。

モーターのエンコーダ・実測速度とタッチセンサ

「モーター A の実際の速さ（%）」はA～DのL/Mモーターの測定値を−100～100%で返します。逆回転は負です。命令した速さとは別の実測値であり、負荷などによる違いを変数・表示・CSVで確認できます。

エンコーダはモーターの累積角度を読みます。タッチセンサは押されているかどうかを読みます。

ブロック	入力・値	使用例
モーターの回転角度 度	A～D。符号付きの累積角度。360度で0へ戻らない。	現在のB角度−開始時のB角度、を計算。
モーターの回転角度を0にする	選んだA～Dの値だけをリセット。	回転量を計測する直前に使う。
タッチセンサが押された／離された	入力1～4。現在の押下状態を判定。	タッチ1が押された、まで待つ。
タッチセンサの値 0／1	押されていれば1、離れていれば0。	押下状態を変数へ保存。

回転量を見て1個のモーターを止める

1. モーターBの回転角度を0にする。
2. Bを20%で回し続ける。
3. Bの回転角度が360以上、まで待つ。
4. モーターBを停止する。

逆回転では角度が負になる場合があります。負の目標を使うか、絶対値で回転量の大きさを比較します。読み取り自体はモーターを動かしたりリセットしたりしません。開始角度を引く方法なら、リセットせずに今回の移動量を求められます。

リセットは物理的な原点復帰ではない

角度を0にするブロックは、その位置を数値上の0にします。モーターやアームを決まった向きまで動かす命令ではありません。回転中でも角度の値を変えるため、計測するタイミングを決めて使います。

タッチで全体を終了する

1. ずっと、の中に「もしタッチ1が押されたなら」を置く。
2. その中へ「プログラムを終了する」を置く。
3. ほかの処理も同じ繰り返しの中で行う。

タッチも「押した瞬間」のイベントではありません。1回ずつ数えたい場合は、押されるまで待つ → 回数を1増やす → 離されるまで待つ、を組み合わせます。

ジャイロセンサ

v0.21.0で追加しました。「センサー」グループで使います。モーターのエンコーダはモーター軸の角度、ジャイロセンサはセンサ自体の回転角度・回転速度を測ります。

追加したブロック

「センサー」グループの「EV3 ジャイロセンサ」にあります。入力ポート1～4を選びます。

ブロック	役割
ジャイロセンサ 1 の角度（度）	回転した角度を数値で返します。逆方向は負。360度で0には戻りません。
ジャイロセンサ 1 の回転速度（度／秒）	1秒あたりの回転角度を数値で返します。逆方向は負になります。

ブロック	役割
ジャイロセンサ 1 の角度／回転速度 が ＜ 90	数値と比較して真偽を返します。＜・＝・＞・≦・≧・≠を選べます。
ジャイロセンサ 1 の角度を0にする	現在の向きを、このプログラム内の0度として記録します。

角度・回転速度は、変数への代入、演算、条件分岐、待機、繰り返し、センサの変化によるイベント、マイブロックの引数、CSV記録、本体画面の数値表示に使えます。PCを切り離した後もEV3本体で読み取ります。

角度と回転速度は、EV3の複合モード GYRO-G&A の2つの値から選びます。同じセンサの角度と回転速度を別の処理で読んでも、モードを切り替えません。センサの種類・モードが違う、または値が不正な場合は、全モーターを停止してエラー音を出し、終了します。接続や認識状態も実機で確認してください。

0度の基準とセンサの再校正

「角度を0にする」は、測定した角度を基準点として覚える操作です。ポートごとに独立し、同じポートを使うすべての並行処理・マイブロックで基準を共有します。プログラムを起動し直すと基準点の記録は初期化されます。開始位置から測りたい場合は、開始時にリセットブロックを置きます。

この操作はセンサ本体の再校正ではありません。「接続・転送」のポート表示はセンサの生値を読むため、リセット後のプログラム内の角度とは異なる場合があります。ポート表示には「角度（センサ生値）」と表示します。プログラム内の角度の確認には、角度ブロックを本体画面に表示するか、CSVへ記録してください。

接続・リセット時はセンサを静止させてください。静止中も角度が変化する場合は、ロボットを止めてセンサのケーブルを挿し直し、値が安定することを確認します。ゼロにする操作だけでは、回転速度のずれを補正しません。正負の向きはセンサの取付方向により変わるため、最初に手で回して確認します。数値はセンサの範囲に依存し、長時間の累積測定ではドリフトや角度カウンタの範囲にも注意してください。

並行する処理がすでに角度を読んでいる途中でリセットすると、イベントが発生する場合があります。初期化が終わってから測定を始めたい場合は、1つのスタートでリセットを行い、その後にメッセージを送って測定側の処理を開始してください。

最初の実機テスト モーターを動かさずに確認

1. ジャイロセンサをEV3の入力ポート1へ接続し、水平な場所で静止させます。
2. サンプルメニューの「ジャイロの角度・回転速度」を選びます。ファイル版は GyroTest.ev3lab です。公開サイトの「使い方・テスト手順」にあるジャイロガイドから保存できます。
3. 「EV3に転送・照合」で転送し、本体で EV3Lab → GyroTest を実行します。「転送して実行」でも確認できます。

4. 実行開始後は1秒間静止させます。角度の基準を0にした後、開始音が鳴ります。
5. 音の後、約10秒間、センサをゆっくり回します。本体画面の上が角度、下が回転速度です。ゆっくり90度程度回す・元に戻す・逆方向へ回す・静止する操作を試します。画面更新や記録のため、測定間隔・所要時間は約0.2秒／約10秒より長くなります。
6. プログラム終了後、再接続します。「接続・転送」の「CSVログを取り出す」でログ名を Gyro にして「EV3からCSVを保存」を押します。保存先プロジェクトは転送時と同じ名前にします。
7. CSVには time_s,item,value の列で angle_deg と rate_deg_s を各50行、合計100行記録します。角度・回転速度は順番に読むため、同一瞬間の測定値ではありません。

同じ Gyro ログ名で再実行すると、前回のCSVを上書きします。比較用のファイルはPCやChromebookへ保存してください。

条件でモーターを止める確認

基本測定が確認できた後、低速で試します。「角度を0にする → タイマーをリセット → モーターを回し続ける → （角度の絶対値が90以上 または タイマーが5秒以上）まで待つ → モーターを停止」の順に組みます。回転方向に左右されないよう絶対値を使い、条件が成立しない場合にも停止する待機時間を設けます。モーターの惰性・負荷・測定周期により、停止後の角度が指定値を超えることがあります。90度での厳密な停止を保証する機能ではありません。

テスト結果の記録

確認項目	記録する内容
環境	EV3 Block Labの版、Windows／ChromeOS、機種、ブラウザの版、EV3ファームウェア、Bluetooth／USB
センサ	ジャイロの型番・接続ポート・取付方向
静止時	角度の変化、回転速度がおおむね0に近いか、観察時間
手で回す	約90度、元に戻す、逆回転、360度超での角度・正負
リセット	角度がほぼ0になるか、他ポートの値が変わらないか
条件とイベント	しきい値を超えたときの分岐・停止・メッセージの動作
並行処理	モーターを動かしながら角度・回転速度の測定とCSV記録が続くか
切り離しと再起動	PCを切り離して実行・本体再起動後に再実行できるか
保存と再利用	.ev3labの保存・読み込み、.ev3blocksの書き出し・読み込み
問題の報告	期待した動き、実際の動き、再現手順、プログラムファイル、CSV、通信ログ、本体表示の写真

未確認の項目は「未確認」とし、成功した条件も記録してください。エラーが出た場合は、再現したプログラムを保存してからブロックや接続を変更します。個人情報を含むファイルは共有前に内容を確認してください。

仕様の参考

- LEGO公式 ジャイロセンサブロック (https://ev3-help-online.api.education.lego.com/Retail/en-us/Gyro.html)：角度・回転速度・比較・リセット。
- LEGO Education Angles and Patterns (https://education.lego.com/en-us/lessons/ev3-robot-trainer/5-angles-and-patterns/)：静止時に角度が変化する場合の確認。
- EV3ファームウェアのデバイス定義 (https://github.com/mindboards/ev3sources/blob/master/lms2012/lms2012/Linux_AM1808/sys/settings/typedata.rcf)：デバイス種類32、複合モード3、2データ。

赤外線センサ

EV3赤外線センサには近さ・リモコン・ビーコンのモードがあります。超音波センサとは別の機器です。

ブロック	入力・結果	注意
近さ 0〜100	入力1〜4。0が近く、100が遠い相対値。	cmではない。
近さが比較値	相対値と数値を比較。	例：近さ＜20。
CHのボタンが選択肢	CH1〜4。選んだ組み合わせに完全一致で真。	送信機と同じCHにする。
CHのボタン番号	下表の0〜11。	番号を変数や比較に使える。
CHのビーコンを検出している	見つかっていれば真。	数値を使う前に判定する。
CHのビーコンの方向	−25〜25。負は左、0は正面、正は右。	度数ではなく相対値。
CHのビーコンまでの近さ	0〜100の相対値。	cmではない。

番号	リモコンの状態	番号	リモコンの状態
0	押されていない	6	左上＋右下
1	左上	7	左下＋右上
2	左下	8	左下＋右下
3	右上	9	ビーコン

番号	リモコンの状態	番号	リモコンの状態
4	右下	10	左上＋左下
5	左上＋右上	11	右上＋右下

リモコンの左は赤、右は青のボタンです。CH1〜4を使えます。ビーコンを探す場合は送信機のビーコンモードをオンにします。

ビーコン未検出は異常ではありません。検出条件は偽、方向は0、近さは100になります。方向0だけで「正面にある」と判断せず、必ず検出条件も確認してください。

近さ・リモコン・ビーコンを交互に読むとモード切り替えが起こり、待ち時間が入ります。機敏な制御では同じモードを使い続けます。接続パネルの表示ではCH1の先頭値だけを表示しますが、プログラムのブロックではCHを選べます。

カラーセンサと超音波センサ

センサの接続先は入力ポート1〜4です。数値を返す丸いブロックと、条件を返すブロックを使い分けます。

ブロック	得られる値・条件	使用例
カラーセンサの色が色	指定色なら成り立つ条件。	もしポート3の色が赤なら音。
カラーセンサの色 番号	0なし、1黒、2青、3緑、4黄、5赤、6白、7茶。	色番号を変数に保存する。
カラーセンサの反射光が比較%	反射光と数値を比較する条件。＜、＝、＞、≦、≧、≠。	反射光が30%未満なら左へ曲がる。
カラーセンサの反射光 %	反射する光の強さ。0〜100。	反射光の数値を演算や変数へ入れる。
超音波センサの距離が比較cm	距離と数値を比較する条件。	距離が15cm未満、まで待つ。
超音波センサの距離 cm	距離0〜255cm。プログラム内では小数部分を切り捨てる。	距離の値を変数へ保存する。

距離で停止する

スタート → LモーターB+Cを20%で回し続ける → 超音波センサ4の距離が15cm未満になるまで待つ → すべてのモーターを停止。この例ではタイヤ直径の設定は不要です。距離を指定して走る命令を使っていないためです。

反射光と色は別のモード

反射光は明るさの数値、色は色の分類です。黒い線を見る場合でも、色の「黒」と反射光のしきい値では判定が異なります。センサを床から同じ高さに保ち、実際の黒と白で値を確認してください。

この版では反射光の校正ブロックと周囲光の読み取りブロックは未対応です。ライントレースのしきい値は、実際の反射光を見て決めます。モード切り替えを頻繁に行うと読み取り待ちが入るため、素早い制御では同じモードを使い続けます。

画面のセンサ表示との違い

接続パネルの値表示は現在のモードを保持し、小数が表示されることがあります。プログラム内の距離ブロックは整数cmなので、画面に13.5cmと出てもプログラム内では13として扱います。

8. 音

音を鳴らす

ブロック	入力と動作
音 Hz を ミリ秒 鳴らす 音量	周波数250～10000Hz、時間1～30000ミリ秒、音量0～100。音の終了を待って次へ進む。

440Hzはラの音です。200ミリ秒は0.2秒です。待つブロックの単位は秒ですが、この音ブロックの時間はミリ秒です。音量0では音は聞こえません。

9. 制御

待機・分岐・繰り返し・終了

条件ブロックは「成り立つ／成り立たない」を表します。命令を入れる欄と、条件を入れる欄を区別します。

ブロック	意味・入力	使用例
秒待つ	0〜60秒。0.5などの小数も使える。	1秒待ってから次の命令へ。
もし条件なら	条件を一度調べ、成り立つ場合だけ中の命令を実行。	もし距離が15cm未満なら停止。
もし条件なら でなければ	成り立つ場合は上の欄、それ以外は下の欄を実行。	赤なら高い音、それ以外は低い音。
回繰り返す	回数0〜1000。中の命令を順に反復。0回は実行しない。	前進→待つ、を10回繰り返す。
ずっと	中の命令を終わりなく繰り返す。	反射光を読み、左右速度を更新する。
条件まで繰り返す	条件が成り立つまで中の命令を繰り返す。最初から成立なら0回。	距離が15cm未満になるまで調べる。
条件の間繰り返す	条件が成り立つ間だけ反復。最初から偽なら0回。	回数が3未満の間、回数を1増やす。
条件まで待つ	条件が成り立つまで本体で読み直して待つ。中に命令を入れる欄はない。	右ボタンが押されるまで待つ。
プログラムを終了する	全モーターを停止し、プログラム全体を終了。	ずっとの中で条件成立時に終了。
このマイブロックを終了する	定義内だけで使用。呼び出し元の次の命令へ戻る。モーターは停止しない。	引数が0なら、このマイブロックを終了。

条件を繰り返し調べるには

「もし」は一度だけの判定です。監視を続ける場合は「ずっと」などの中に入れます。条件待ちは、センサやボタンなど変化する値を条件に使います。固定の偽条件では先へ進めません。

センサを使わない条件付き繰り返しは、事前検証で1000回以内に終わることを確認します。「ずっと」に同じ1000回の上限はありません。

次の命令へ進む時点

次の命令へ進むタイミング

命令	次へ進む時点
回し続ける／左右の速さで回し続ける	回転開始の命令を出した直後。
角度指定／時間指定／距離移動／旋回	指定した動きの完了後。
音を鳴らす	指定した音の完了後。
停止	停止命令を出してから次へ進む。

例：左30%・右15%で回し続ける → 1秒待つ → 全モーター停止。この並びなら約1秒間カーブします。時間指定のない連続回転だけを置いても、プログラム終了時には停止します。

PID制御の入力・計算・リセット

PIDの入力と出力

「制御」グループの「PID制御」にあります。PIDブロックは数値を返す丸いブロックです。単独ではモーターを動かしません。

入力	意味
PID名	初期値はPID1。同じ名前は同じ制御器、違う名前は別の制御器です。1〜40文字、全体で16種類まで。
目標値	到達させたい値。数値・変数・演算・引数を使えます。
測定値	センサなどで得た現在の値。目標値と同じ単位にします。
kp（比例ゲイン）	現在の偏差に対する比例ゲイン。初期値1。
ki（積分ゲイン）	偏差の積分に対する積分ゲイン。初期値0。
kd（微分ゲイン）	偏差の変化速度に対する微分ゲイン。初期値0。
出力	PID計算の結果を数値で返します。変数・モーターの速さ・表示・記録などへつなぎます。

符号を含めた小数のゲインを使えます。モーターの向きや目標値の単位により、適切なゲインと出力の単位は変わります。

計算式と時間

偏差 e ＝ 目標値 − 測定値

偏差の積分 I ＝ 前回の積分 ＋ 今回の偏差 × 経過秒数

偏差の変化速度 D ＝（今回の偏差 − 前回の偏差）÷ 経過秒数

出力 u ＝ kp（比例ゲイン） × e ＋ ki（積分ゲイン） × I ＋ kd（微分ゲイン） × D

経過秒数はEV3本体のミリ秒時計で自動計測します。表示用タイマーのリセットでは変わりません。初回は積分と微分を0とし、P項だけを返します。同じミリ秒内の再評価は、ゼロ除算や積分の二重加算を避けるため、前回出力を返します。入力やゲインを変えても、次のミリ秒までは同じ結果になる場合があります。前回計算から60秒を超えて空いた場合は、古い積分を消してP項から再開します。

「ずっと」の中で計算し、例えば0.02秒待つ構成から試します。これは厳密な20ms周期の保証ではなく、センサ取得やCSV保存などの処理時間も加わります。実際の経過時間を使って積分・微分を計算します。D項は測定の揺れや目標値の急変で大きくなるため、最初はI・Dを0にしてPだけで確認してください。

リセットと状態の共有

「PID PID1 をリセットする」は、積分・前回の偏差・前回時刻・出力を初期化します。次の計算はP項だけです。モーターを動かしたり止めたりする命令ではありません。プログラムの実行開始時にも、すべてのPIDを初期化します。

同じPID名は、別のブロック・マイブロック・並行処理でも状態を共有します。名前が同じ処理は内部で順番に計算しますが、複数の処理が別々の目標値を渡すと制御に干渉します。1つの制御器の更新担当は1つに決め、出力を変数に保存してほかの処理で使ってください。左右など別の制御には、異なるPID名を付けます。マイブロックを読み込んだ際も、同じ名前のPIDは共有するので確認してください。

出力をモーターへ渡す

このPIDには出力上限・積分上限や自動調整機能はありません。モーターの速さに渡す場合は「値を−100から100の範囲に収める」ブロックを使います。最初は−20〜20など小さい範囲に収めて実機を確認します。

例：PIDをリセット → ずっと［出力にPIDの計算結果をセット → 出力を−20〜20に収めてモーターを回し続ける → 0.02秒待つ］。

連続回転の速さは負なら逆転、0なら停止です。角度・時間を指定する有限動作では速さ0は使えません。センサや制御の用途に合った停止条件を組み合わせます。

出力を範囲内に収めても、PID内部の積分値は増え続ける場合があります（積分ワインドアップ）。この版に自動的な積分抑制はありません。目標変更や試行の前にリセットし、ki（積分ゲイン）は小さい値から確認してください。実行中に不正な数値や計算範囲超過を検出した場合は、全モーター停止・エラー音・プログラム終了の処理へ進みます。

モーターを動かさない実機テスト

サンプル	確認方法	CSV名
バッテリー電圧・電流	BatteryTestを転送・実行。本体画面の上がV、下がA。約0.5秒待ちながら各20回を記録します。	Battery
PID計算（モーターなし）	PIDTestを転送・実行。目標10、測定4、kp=1、ki=0.5、kd=0.1で10回計算。初回出力は6、その後は積分の分だけ増えます。	PID

測定間隔には表示・記録の処理時間も加わります。PIDTestの測定値は固定値で、実際の機械をフィードバック制御するサンプルではありません。ki=0、kd=0にして出力が6のままになること、リセット後に初回の値へ戻ることを比較できます。

終了後に再接続し、「接続・転送」の「CSVログを取り出す」で、上表のログ名を指定して保存します。同じログ名で再実行すると前回のCSVを上書きします。成功・不具合のどちらも、アプリの版、端末とブラウザ、EV3のファームウェア、Bluetooth／USB、プログラム・CSV・通信ログを記録してください。

10. 演算

四則演算・比較・論理

数値のブロックは数値欄へ、比較・論理の条件ブロックは「もし」などの条件欄へ入れます。

ブロック	意味	例と結果
数値	直接数を入力。負数・小数も使える。	−20、0.5、360
＋	2つの数を足す。	3＋2 → 5
−	左の数から右の数を引く。	3−5 → −2
×	2つの数を掛ける。	0.5×10 → 5
÷	左の数を右の数で割る。0では割れない。	7÷2 → 3.5
＜／≦	左が右より小さい／以下。	3＜5 → 真、5≦5 → 真
＞／≧	左が右より大きい／以上。	3＞5 → 偽、5≧5 → 真
＝／≠	2つの数が等しい／異なる。	5＝5 → 真、5≠5 → 偽
かつ	両方の条件が成り立つ。	距離＜15 かつ 反射光＜30
または	少なくとも一方が成り立つ。	赤 または タッチが押された
ではない	条件の成り立つ／成り立たないを反転。	右ボタンが押されている、ではない
真／偽	固定の条件を選択。	もし真なら、は常に実行する。

小数計算の精度と整数への変換

v0.15.0以降は、数値関数の有無にかかわらず、四則演算・変数・引数で小数を保持します。EV3の32ビット浮動小数点で計算するため有効数字は約7桁で、0.1などには丸め誤差があります。

小数を加算した結果を「＝」で比べると意図に合わないことがあります。目標に到達したかは「≧」や「≦」などを使い、必要なら許容範囲で判定してください。

速さ%、角度、回数など整数を要求する命令では、小数部分を0に近い方向へ切り捨てます。例：20.9→20、−20.9→−20。秒は小数のままミリ秒へ換算します。

この版の演算は数値が対象です。乱数・余り・四捨五入などは第10章、文字列は第13章、数値リストは第13章で説明します。

数値関数

「数値 の 絶対値」のプルダウンを開いて関数を選びます。値には数値・変数・引数・演算・センサ値を入れられます。

選択肢	意味・使える値	例と結果
絶対値	符号を取り除いた大きさ。	−7.25 → 7.25
切り下げ	その値以下で最大の整数。	2.3 → 2、−2.3 → −3
切り上げ	その値以上で最小の整数。	2.3 → 3、−2.3 → −2
平方根	掛けると元の数になる非負の数。入力は0以上。	9 → 3
sin	角度の正弦。入力の単位は度。	30 → 約0.5
cos	角度の余弦。入力の単位は度。	60 → 約0.5
tan	角度の正接。90度＋180度の整数倍は不可。	45 → 約1
asin	逆正弦。入力−1〜1、結果は度。	0.5 → 約30
acos	逆余弦。入力−1〜1、結果は度。	0.5 → 約60
atan	逆正接。結果は度。	1 → 約45
ln	自然対数。入力は0より大きい数。	1 → 0
log	底10の対数。入力は0より大きい数。	1000 → 約3
e^	eの指定乗。eは約2.71828。	0 → 1
10^	10の指定乗。	−2 → 約0.01

入力できない値と計算範囲

負の平方根、0以下の対数、範囲外のasin／acosなどはエラーになります。転送前に分かる場合は生成を止め、センサなどの値で本体実行中に異常が分かった場合は全モーターを停止して終了する処理を入れています。

計算結果がEV3の単精度で表せる範囲を超える場合もエラーです。センサ値や変数を入力するときは、先に「もし」で範囲を確認すると処理を理解しやすくなります。

例

モーターBの回転角度の絶対値が360以上、まで待つ → Bを停止。回転方向によって角度が負になっても、回転量の大きさで止められます。

乱数・余り・丸め・範囲制限

「演算」グループに、余り、乱数、四捨五入、最小・最大、値を範囲内に収めるブロックを追加しました。値には変数、引数、演算、センサを使えます。

ブロック	意味と範囲	例
AをBで割った余り	小数も扱う。0では割れない。余りの符号は割る数と同じ。	−7を3で割った余り → 2
AからBまでの整数乱数	下限・上限を含む。−32767〜32767。小数の上下限は0方向へ切り捨て。下限≦上限。	1〜6 → サイコロ
Aを四捨五入	0.5を足して切り下げる。	1.5 → 2、−1.5 → −1
AとBの小さい方	小さい数を返す。	20と35 → 20
AとBの大きい方	大きい数を返す。	20と35 → 35
Aを下限から上限の範囲に収める	下限未満は下限、上限を超えれば上限。範囲内ならその値。下限≦上限。	120を−100〜100へ → 100

モーターの速さを範囲内に収める

センサや計算で求めた速さを「−100から100の範囲に収める」へ入れ、その結果を連続回転の速さへ渡します。有限の角度・時間指定では速さ0を使えないため、必要なら「もし」で0を別に扱います。

回数から周期を作る

「回数を4で割った余り」が0ならランプを緑、でなければオレンジにする、などの周期的な処理に使えます。回数を1増やす命令も繰り返しの中へ置きます。

乱数は本体で決まる

実行時にEV3の乱数命令で選びます。ブラウザで表示した1つの数を固定して転送する方式ではありません。どの値が選ばれても、モーターや待ち時間の入力範囲に収まるように上下限を決めます。

11. 画面

「画面」グループで使います。画面は白黒178×128ドットで、左上が横0・縦0。右へ横、下へ縦の値が増えます。全ての表示命令は描画後すぐ次へ進みます。

ブロック	入力と動作
本体の画面を消す	画面全体を白くする。
本体に文字を表示	固定文字1〜120文字。日本語可。大きさは小12・中16・大24ドット。長い文字は178ドット幅で折り返す。
本体に画像を表示	えがお・かなしい顔・びっくりした顔・ハート・チェック・ばつ・上下左右矢印の10種類。48または96ドットの正方形。
本体に数値を表示	変数・センサ・演算を差し込む。小数点以下0〜3桁。符号・小数点込みで先頭10文字まで。

文字・画像の位置は横0〜177、縦0〜127ですが、全体が画面に収まる必要があります。数値の表示欄は小80×9、大160×19ドットで、欄ごと収まる位置を指定します。

表示部分を白くしてから描き直します。それ以外は残ります。文字数や位置を変えたとき、前の文字が残る場合は「画面を消す」を先に置きます。複数の並行処理からの描画は1ブロック単位で順番に行います。同じ場所は後から描いた内容になります。

表示を確認するサンプル

上部「サンプル → 本体画面の表示」：画面を消す → 横0縦4に「こんにちは EV3」 → 横64縦34にえがお48ドット → 横48縦94にタイマーの値 → 5秒待つ。

このサンプルのタイマー表示は、表示命令を実行した時点の値です。待っている間は更新しません。更新するには「ずっと［タイマーを表示 → 0.1秒待つ］」にします。センサ値の表示も同じです。

日本語は転送時にブラウザのフォントで白黒画像へ変換し、描画命令を実行ファイル内に埋め込みます。追加画像ファイル不要で、本体だけで表示できます。字形は作成端末のフォントで異なります。文字列変数の動的表示・画像ファイル読み込みは今回の対象ではありません。

表示直後にプログラムが終了すると見えにくい場合があります。観察用の待ち時間を入れます。文字・画像を増やすと実行ファイルが大きくなるため、64KB上限にも注意してください。表示品質と更新速度は実機確認を続けています。

12. ランプ

「ランプ」グループで、緑・赤・オレンジの点灯や点滅を指定できます。ブラウザを切り離してもEV3内で動作します。

ブロック	入力・動作	使い方
本体のランプを 色 に 点灯	緑・赤・オレンジ。次の命令へすぐ進む。	動作の開始や正常状態を伝える。
本体のランプを 色 に 点滅	選択した色で点滅。次の命令へすぐ進む。	注意や処理中を知らせる。
本体のランプを 色 に ゆっくり点滅	選択した色でゆっくり点滅。	点滅との違いを実機で確かめる。
本体のランプを消す	ランプを消し、次へ進む。	必要な表示を終えたとき。

色と点滅を順番に試す

1. スタート → 緑に点灯。

2. 1秒待つ → 赤に点灯。

3. 1秒待つ → オレンジに点滅。

4. 1秒待つ → 緑にゆっくり点滅。

5. 1秒待つ → ランプを消す。

点灯命令そのものは待ちません。すぐ終了すると表示を確認しにくいため、待つ命令などを続けます。プログラム終了後はEV3本体の通常のランプ表示に戻ります。

3色は本体のランプの色です。カラーセンサで測る色や、パソコン画面のブロックの色とは別のものです。

13. データ

文字列と文字列変数

「データ」グループで、文字や説明文を扱います。数値用の通常変数とは分けて、文字列変数のブロックを使います。

ブロック	意味	例
文字列 テキスト	文字を直接入力する。	こんにちは
AとBをつなぐ	2つの文字列を連結。	回数= と 数値を文字列にする
文字列の文字数	日本語も1文字として数える。結合文字は別に数える。	こんにちは → 5
文字列のN番目の文字	1から数える。文字数を超えると空文字。Nは1〜120。	こんにちは の2番目 → ん
AにBが含まれる	部分文字列が見つかれば真。空文字は含まれる。	EV3 Lab に EV3 → 真
AとBが同じ	文字列全体の一致。	赤 と 赤 → 真
数値を文字列にする	数値を有効数字約7桁で文字へ変換。	12.5 → 文字の12.5
文字列を数値にする	半角の符号・数字・小数点・指数表記。空白なし。不正な文字列はエラー。	12.5、-2.5、1e2
文字列変数 名前 を値にする	同じ名前の文字列変数へ保存する。	メッセージを「終了」にする。
文字列変数を取得	名前だけの丸いブロック。数値変数とは別の型。	CSVの値や連結に使う。

名前と長さの上限

文字列変数は各ブロックへ名前を入力し、同じ名前で共有します。1〜40文字の名前で16個まで。プログラムを開始すると空文字に戻ります。文字列はUTF-8で120バイトまで（日本語は通常40文字まで）。連結後も上限を超えるとエラー終了します。

文字列はモーターの速さなどの数値欄には直接入りません。必要なら数値に変換します。この版のマイブロックの引数は数値用です。文字列を引数として渡す機能は含みません。

数値リストと集計

「データ」グループのリストは、複数の数値を順番に蓄えるためのものです。同じ名前を各ブロックに入力すると、共通のリストを使います。文字列のリストはこの版には含みません。

ブロック	意味	注意
リストを空にする	すべての項目を削除する。	測定前の初期化など。

ブロック	意味	注意
リストに値を追加	末尾へ数値を1個追加。小数も可。	最大1000項目。
N番目を値にする	指定した項目を置き換える。	存在する項目だけ指定。
N番目を削除	指定した項目を削除し、後ろを詰める。	番号は1から。
N番目を取得	その位置の数値を返す。	存在しない番号はエラー。
リストの項目数	現在の数。空なら0。	繰り返し回数などへ。
リストに値が含まれる	同じ数値が1個以上あれば真。	小数の一致は丸め誤差に注意。
リストの合計	全項目を足す。空なら0。	計算範囲を超えるとエラー。
リストの平均	合計÷項目数。	1項目以上必要。
リストの最小値／最大値	全項目から小さい／大きい値を返す。	1項目以上必要。

反射光を10回測って平均を記録する

1. リスト「測定」を空にする。CSVログLogを開始する。

2. 10回繰り返す［カラーセンサの反射光をリスト「測定」に追加 → 0.1秒待つ］。

3. 項目「反射光の平均」、値「リスト測定の平均」をCSVに記録する。

4. CSVログを終了し、停止後にPCへ取り出す。

名前は1〜40文字、リストは16個まで。プログラム全体とマイブロックから共有します。実行開始時は空になり、前回の実行値は引き継ぎません。残す場合はCSVに記録します。

14. 記録

CSVをEV3本体に記録する

「記録」グループのブロックで、変数・演算結果・センサ値・文字列をEV3本体に保存します。PCやChromebookを切り離して実行しても記録できます。

ブロック	入力・動作	注意
CSVログ Log の記録を開始する	プロジェクトのフォルダ内にLog.csvを作り、開始時刻と行数を初期化。	同じ名前のCSVは上書き。
項目 名前 値 をCSVに記録する	開始からの秒数、項目名、値を1行記録。値は数値か文字列。	開始後、終了前に使用する。
CSVログの記録を終了する	記録の受付を終了する。次の命令へ進む。	以後の記録には再び開始が必要。

名前と保存先

ログ名は半角英数字・ハイフン・アンダースコアで1〜31文字。拡張子は入力しません。プロジェクト名がClass1、ログ名がLogなら、本体のClass1フォルダ内にLog.csvが保存されます。実行ファイルの名前とは別に決めます。

CSVの3列

列	意味	例
time_s	記録開始からの秒数。表示用タイマーをリセットしても別の時計として進む。	0.100
item	記録する値の名前。日本語を使える。	反射光
value	その時点の値。小数は有効数字約7桁。	42.5

回数を記録する

1. CSVログLogの記録を開始する。

2. 回数に0をセットする。

3. 10回繰り返す［回数を1増やす → 項目「回数」、値「回数」を記録 → 0.1秒待つ］。

4. 記録を終了する。

1回の開始につき5000行までです。複数の値は項目名を変えて順番に記録するため、同一瞬間の測定にはなりません。記録に処理時間がかかるので、厳密な周期の計測には向きません。

1行ごとにファイルを閉じますが、書き込み中の電源断では最後の行が不完全になる場合があります。記録を終え、本体を停止してから取り出してください。

CSVをPC・Chromebookへ取り出す

本体だけで実行した後でも、再接続してCSVを保存できます。取得したCSVを表計算ソフトで開き、表やグラフに使えます。

取り出す手順

1. 本体のプログラムを停止する。実行中の取得は受け付けません。
2. BluetoothまたはUSBでEV3に再接続する。
3. 画面左上のプロジェクト名を、記録したフォルダの名前に合わせる。
4. 「接続・転送」パネルの「CSVログを取り出す」を開く。
5. ログ名を拡張子なしで入力する。例：Log。
6. 「EV3からCSVを保存」を押し、PC／ChromebookへLog.csvを保存する。

本体の記録は残る

取り出す操作ではEV3上のCSVは削除しません。再度取り出すこともできます。ただし、本体で同じログ名の「記録を開始する」を実行すると、以前のCSVは上書きされます。残す場合は先に取り出すか、別のログ名で記録してください。

日本語と表計算ソフト

PC側へはUTF-8 BOM付きで保存します。カンマ、改行、引用符を含む文字列はCSV用に囲み、引用符を重ねて保存します。数式と解釈されやすい「=」「+」「-」「@」で始まる項目名や文字列には、先頭にアポストロフィを付けます。数値の負の値には付けません。

取り出せない場合

症状	確認すること
実行を停止するよう表示	記録しているプログラムを停止してから再取得。
ファイルが見つからない	プロジェクト名、ログ名、今回のプログラムを実行したかを確認。
対応CSVではない	EV3 Block Labが生成したCSVを指定する。
サイズ上限	PCへの取り出しは2MBまで。通常の5000行の記録はこの範囲内。

すぐに試す

「その他 → CSV・リストのサンプル」は、回数1〜10、説明文、平均5.5をLog.csvに記録します。モーターは動かしません。転送し、本体で実行してから取り出す流れを最初に確認できます。

15. 変数

変数は、途中で変わる数値を名前を付けて保存する場所です。プログラム全体と各マイブロックから同じ通常変数を参照できます。

操作・ブロック	意味	使用例
変数をつくる	「変数」グループのボタンで名前を入力。1〜40文字。	「回数」「開始角度」「目標距離」など。
変数名だけのブロック	その変数の現在の数値を取り出す。	「回数」を比較ブロックへ入れる。
変数を値にセット	選んだ変数へ数値を保存。以前の値は置き換わる。	回数に0をセット。
変数を値ずつ変える	選んだ変数へ指定値を加算する。負の値で減算。	回数を1ずつ変える。回数を−1ずつ変える。

変数の選択と初期値

取得ブロックは変数名ごとにグループ内へ並びます。セット・増減ではプルダウンで変数を選びます。通常変数は実行開始時に0になりますが、学習用には先頭で明示的に初期値を設定すると分かりやすくなります。

3回だけ音を鳴らす

1. 回数に0をセットする。

2. 「回数＜3」の間繰り返す。

3. 繰り返しの中に、音を鳴らす → 回数を1ずつ変える → 0.5秒待つ、を入れる。

変数と引数の違い

項目	通常変数	マイブロックの引数
値が決まるとき	セット・増減の命令で変更。	呼び出すときに渡す。
使える範囲	プログラム全体。	その引数を定義したマイブロック内。
同じ名前の値	同じ通常変数を共有。	マイブロックの呼び出しごとに独立。
直接の書き換え	セット・増減が使える。	この版ではセット・増減の対象にしない。

通常変数は32個までです。数値は小数も保持します。文字列変数と数値リストは「データ」グループで別に扱います。引数だけに使う名前は通常の変数一覧には表示されません。

16. マイブロック

何度も使う命令を定義へまとめ、呼び出して使います。「定義」と「呼び出し」は役割が異なります。

操作・ブロック	意味	注意
マイブロックを作る	名前と必要な引数をフォームで入力する。	名前1〜60文字。引数8個まで、各1〜40文字。
定義 名前 引数	処理の内容を書く場所。	スタートには接続しない。定義の中に命令をつなぐ。
名前 引数欄	定義した処理を呼び出す命令。	マイブロックのグループから取り出してつなぐ。
引数名だけのブロック	今回の呼び出しで渡された数値を取り出す。	「引数」グループに表示。その定義の中だけで使う。
編集	名前や引数を変更する。	グループの編集ボタンや定義のメニューを使う。使用中の引数は先に外してから削除。
このマイブロックを終了する	残りの処理を行わず、呼び出し元へ戻る。	全体終了やモーター停止とは異なる。

引数を変えて同じ処理を使う

1. マイブロック「移動」を作り、引数「距離」「速さ」を追加する。

2. 呼び出し側で左右モーターとタイヤ直径を設定し、定義にポート欄のない「速さ%で距離cm前に進む」を置き、それぞれの入力欄に引数を入れる。

3. スタートの下でタイヤ直径を設定する。

4. 「移動 距離20 速さ30」、次に「移動 距離10 速さ20」を呼び出す。

別の作品へ持ち出す

「その他 → マイブロックを書き出す」で集めて保存し、別作品で読み込みます。同名の定義は上書きしません。同名の通常変数は共有されます。定義は32個までです。戻り値は数値に対応します。文字列・リストの戻り値と再帰呼び出しには未対応です。

数値を返すマイブロック

「マイブロックを作る」の「戻り値」で「数値を返す」を選びます。「＋ 戻り値を追加」で名前を付け、最大8個まで設定できます。名前は1〜40文字で、同じ定義内では重複させません。定義の下部に各名前の「返す値」が現れます。初期値は0です。ここに数値、引数、変数、センサ、演算、別の数値を返すマイブロックを入れます。

戻り値が1個の場合、数値を返す呼び出しは丸い形です。変数の「○にする」、演算、モーターの速さ・角度、センサの比較、CSVの値など、数値の入力欄に入れます。呼び出したときに定義内の命令を実行し、最後に「返す値」を計算して呼び出し元へ返します。定義を置くだけでは実行されません。

例：「平均」を作り、引数を「値1」「値2」、返す値を「（値1＋値2）÷2」にします。「平均 440 880」を音の周波数に入れると、660Hzの音が鳴ります。サンプル「マイブロックの戻り値」（MyBlockReturn.ev3lab）はモーターを動かさずに試せます。

ブロック・設定	動作	注意点
戻り値：なし	従来どおり命令をまとめる。	呼び出しを命令の並びにつなぐ。
戻り値：数値を返す（1個）	定義内の命令の後、「返す値」を計算する。	丸い呼び出しを数値欄に差し込む。小数も使える。
戻り値：数値を返す（2〜8個）	1回の呼び出しで全ての戻り値を計算し、選んだ変数へ代入する。	呼び出しを命令の並びにつなぎ、各名前の右側で受け取り先の変数を選ぶ。

ブロック・設定	動作	注意点
値を返してこのマイブロックを終了する	その場で数値を返し、定義の残りを実行しない。	数値を返す定義の中だけで使用。条件分岐・繰り返しの途中でも使える。
このマイブロックを終了する	値なしで呼び出し元へ戻る。	戻り値なしの定義で使う。

途中で値を返す例：引数が0未満なら「引数×−1を返してこのマイブロックを終了する」。それ以外は定義末尾の「返す値」に引数を入れます。

引数・戻り値は呼び出すたびに別々です。並行処理から同じ定義を呼んでも混ざりません。通常変数、動きの共通設定、PIDなどの共有状態を定義内で変更すれば、ほかの処理からも変更が見えます。同じ機器へ同時に命令しないようにしてください。再帰呼び出し（自分自身へ戻る呼び出し）は使えません。

戻り値の有無・個数を変更すると呼び出しの形が変わるため、既存の呼び出しブロックを先に外して削除し、変更後に取り出し直します。定義の命令は保持します。保存ファイルとマイブロック集には、数値を返す定義・呼び出し・返す値も保存されます。

複数の戻り値を変数に受け取る

例：「和と差」を作り、引数を「値1」「値2」、戻り値を「和」「差」にします。返す値はそれぞれ「値1＋値2」「値1−値2」です。呼び出しをスタートの下につなぎ、引数を440、220にして、「和 → 合計」「差 → 差分」と受け取り先を選びます。合計には660、差分には220が入ります。

定義は1回だけ実行します。センサを読み取った結果を複数返す場合も、戻り値ごとに定義を繰り返し実行する方式ではありません。ただし、異なるセンサの読み取り自体は順番に行うため、厳密な同時測定ではありません。

受け取り先は通常の変数です。返す値の計算をすべて終えてから、表示順に変数へ代入します。同じ変数を複数の受け取り先に選ぶと、最後の値が残るため、通常は別々の変数を選んでください。変数への代入は、1回の呼び出しの分をまとめて行います。

複数の値を途中で返す場合は、マイブロックの一覧にある「和 [値] 差 [値] を返してこのマイブロックを終了する」のように名前が並ぶブロックを使います。数と名前を定義に合わせます。1個用の終了ブロックでは、複数の戻り値を省略して返せません。

名前だけを変更する場合は、定義・呼び出し・同じ定義内の終了ブロックのラベルを更新します。個数の変更では、呼び出しを先に削除して変更後に置き直してください。削除する戻り値につながる計算ブロックも先に外します。保存・再読み込みとマイブロック集の書き出し・読み込みに対応します。

実機テスト

サンプルを転送・照合し、本体から起動して660Hzの音を確認します。引数を変更して計算結果が変わること、切断後と電源再投入後も動くことを確認してください。自動テストでの成功は、EV3実機での確認を意味しません。「複数の戻り値（和と差）」では、本体画面の上に660.0、下に220.0を表示し、660Hzの音を鳴らしてから2秒待ちます。引数の変更、切断後と電源再投入後、複数スタートからの呼び出しも確認してください。今回の複数の戻り値機能は実機確認待ちです。

17. 引数

引数は、マイブロックの呼び出しで渡された数値を受け取るためのブロックです。「引数」グループから選び、定義の中の数値入力へ入れます。引数名だけを表示します。通常の変数とは別の一覧です。

同じ名前の引数を使う

複数の定義に「距離」があっても、一覧には1個だけ表示します。どちらの定義でも使えますが、値が共有されるわけではありません。呼び出し欄の入力は横並びで、0をクリックして数値を変えたり、変数や演算を差し込んだりできます。

18. 授業の作例・サンプルと実機テスト

授業で使う組み合わせ

以下は実行順を文字で表した例です。矢印の順に命令をつなぎ、繰り返しや分岐の内部へ必要な処理を入れます。

障害物の手前で止まる

スタート → B+Cを20%で回し続ける → 超音波センサ4の距離 <15cmまで待つ → 全モーター停止 → 音。最初は車輪を浮かせて、距離と停止を確認します。

色で音を変える

スタート → ずっと［もしカラーセンサ3の色が赤なら、880Hzを200ms鳴らす。でなければ、440Hzを200ms鳴らす］。この並びは赤を検出した瞬間だけでなく、赤の間は繰り返して鳴ります。

反射光による簡単なライントレース

スタート → 動きに使うモーターを左B・右Cにする → ずっと［もし反射光 <しきい値なら、左の速さ10%・右の速さ30%で回し続ける。でなければ、左の速さ30%・右の速さ10%で回し続ける］。黒い線のどちら側を走るかで左右を入れ替えます。しきい値は実際の黒・白を測って決めます。

マイブロックを条件で途中終了する

定義「移動 距離 速さ」：もし距離≦0なら、このマイブロックを終了する。その下で、指定した距離と速さを使って前進する。呼び出し元は戻った後の命令を続けられます。

全体終了とモーター停止を区別する

やりたいこと	使う命令	その後
Bだけ止めたい	モーターBを停止	ほかの回転とプログラムは続く。
すべての回転を止めたい	すべてのモーターを停止	次の命令は続く。
プログラム自体を終えたい	プログラムを終了する	呼び出し元にも戻って続けない。
今のマイブロックを抜けたい	このマイブロックを終了する	呼び出し元の次へ。回転は止めない。

本体に転送した後はブラウザでセンサを監視しなくても動きます。別のスタートで計測を並行したり、センサ条件からイベントを起動したりできます。第3〜4章を参照してください。

サンプル一覧と結果の報告

サンプル名	接続・内容	CSV名
複数の戻り値（和と差）	モーターなし。和660・差220を1回で受け取り、画面表示と音で確認。	なし
停止方法を比較	モーターB。惰性・ブレーキ・保持を順番に比較。	なし
左右別速度で角度・時間指定	同じ種類のB+C。30%対15%で速い側180度、逆回転で0.5秒。	なし
速度・パワーと実測値	モーターB。速度20%・30%、パワー20%と実測値。	Power
走行と計測を並行	B+Cと超音波2。2秒の走行と20回の計測。	Parallel
タッチで終了（イベント）	B+Cとタッチ1。押下で全プログラム終了。	なし
メッセージで測定	超音波2。10回の記録完了を待ってランプを変更。	Message
共通設定で前進	左B・右Cを共通設定し、直径56mm・20%・20cm前進。	なし
途中で左右モーター変更	B+Cで前進後、旧組を停止しA+Dへ切替えて後退。配線を確認。	なし
共通設定とマイブロック	呼び出し側で左右と直径を設定し、ポート欄のない前進の定義を利用。	なし
バッテリー電圧・電流	本体V・Aの表示と各20回の記録。	Battery
PID計算（モーターなし）	目標10・測定4から出力を10回表示・記録。	PID
本体画面の表示	モーターなし。日本語・えがお・数値を5秒表示。	なし
ジャイロの角度・回転速度	ジャイロ1。開始時に角度を0にし、角度・回転速度を約10秒間表示・各50回記録。モーターなし。	Gyro

不具合を報告するとき

作品を.ev3labで保存し、通信ログを開いてログも保存します。端末・OS・ブラウザ・EV3のファームウェア、接続方法、使ったポートとセンサ・モーターの種類、操作手順、期待した動きと実際の動きを添えてください。実行時の本体エラーや停止までの様子も役立ちます。

本体の画面表示、停止方法、並行実行、USB接続などは、自動試験の合格だけで全実機環境の動作を保証しません。最初のモーターテストは車輪を浮かせ、停止操作が使える状態で確認します。

アプリ本体はv0.25.0。自動試験とブラウザ画面の確認を行った状態です。左右モーターの共通設定を含む追加機能の実機確認は所有者が担当し、未実施の項目は確認待ちとして扱います。実機テストへの参加とフィードバックを歓迎します。

左右モーター共通設定の実機テスト

所有者が担当する実機テスト

確認項目	確認方法
配線と基本走行	左右の実配線に合わせ、低速で前進・後退。B+C以外と左右逆の設定も試す。
旋回	直径・トレッドを実測して、右／左スピン・右／左ピボットを確認。
曲線と左右別速度	ステアリングと左右別速度それぞれの連続・角度・時間を確認。
設定の反復と変更	同じ設定では連続回転を中断しないこと、別の組への切替時には旧組が止まり、その後は新組で動くこと。
再利用と並行処理	呼び出し側で設定したマイブロック、設定後にメッセージで動作を始める例。
単独モーター	共通設定に含まれないアーム等を、従来のポート欄で動作・停止できること。
未設定エラー	実行順が定まらない並行例で、未設定なら番号23とエラー音で停止すること。
単体実行	切断後・本体電源再投入後も先頭で設定を実行して動くこと。

自動テストとブラウザ確認の成功は、実機確認済みを意味しません。OS・ブラウザ・本体FW・配線・モーター種別と結果を記録してください。生成する実行プログラムの上限64KBは従来どおりです。多くの異なる配線設定と長い補正走行を組み合わせて超える場合は、プログラムを分けてください。

19. エラーと制限

画面下の赤い生成エラーは、接続不良とは限りません。転送ボタンが使えない場合は、まず生成結果を確認します。

表示・症状	確認と対応
距離で動かす前に直径を設定	スタートの後、移動命令より前に直径ブロックを置く。
旋回の前にトレッドを設定	直径とトレッドの両方を実行してから旋回する。
数値・条件の空欄	必要な形の数値・条件ブロックを入れる。
開始ブロックが多すぎる	スタート・センサイベント・メッセージ受信は合計16個まで。未接続の試作ブロックは残せる。
接続済みだが転送できない	名前の入力、生成エラー、別の通信中でないかを確認。
本体に転送できたが動かない	照合だけなら本体で実行する。ポートと今回のファイル名も確認。
VM PROGRAM VALIDATIONなど	作品と通信ログ、本体の表示を保存し、開発者へ共有する。
古い画面のまま	ページを再読み込みし、v0.25.0を確認して保存ファイルを開き直す。

この版の主な上限

作品内のブロック1000個、通常変数32個、マイブロック32個、1定義の引数8個。生成する実行ファイルは64KBまでです。複雑な入れ子や大きな繰り返しでは、このほかに検証上限で生成できない場合があります。文字列変数と数値リストは各16個まで、リストは1000項目までです。文字列はUTF-8で120バイトまで、CSVは1回の開始につき5000行までです。

センサ未接続・種類違い・不正値などを本体で検出した場合は、全モーター停止と低い音を伴って終了する場合があります。モーターの負荷や通信切断は実物で確認してください。

旧ブロックと未対応機能

以前保存したミリ秒待機・旧L/Mモーター・演算のプルダウン型などは互換用です。新しく作る場合は現在のカテゴリから取り出します。反射光校正専用ブロック、周囲光の読み取りブロック、音声ファイル再生、学年別表示、文字列を引数に渡す機能、文字列リストは未対応です。文字列変数を本体画面へ動的に表示する機能も含みません。

参照先と改訂

アプリ：https://ev3-block-lab-prototype.davinichi-9576.chatgpt.site/
ソース：https://github.com/davinichi/ev3-block-lab
本書はv0.25.0のapp.js、ev3.js、data.js、library.js、sensor-monitor.js、motor-features.js、parallel.js、movement.js、display.jsを基に更新しました。今回の戻り値対応は自動テストで確認し、実機確認は所有者が担当します。マニュアルは2026年11月1日更新。

20. EV3 Classroomとの機能比較と開発の目的

この比較はEV3 Block Lab v0.25.0の実装と、LEGO Education・正規代理店アフレルの公開資料に基づきます。ClassroomはOS・版で違いがあります。Classroom欄は公開資料に記載された機能の比較で、現在の各ストアでの提供状況を保証するものではありません。「実装済み」は同じ精度や完全互換の保証ではありません。

機能	EV3 Classroom	EV3 Block Lab v0.25.0
利用方法	インストール型。ChromeOSはAndroid版。	ブラウザ型。インストール不要。
モーターの速度・角度・回転数・時間	対応	実装済み。L・M／1個・2個別に表示。
2モーター・左右別速度・旋回	対応	同期・ステアリング・左右別速度を実装。動きの左右ポートは共通設定。
停止方法・エンコーダ・実測速度	対応	実装済み。停止方式には動作差がある。
超音波・カラー・タッチ・赤外線	対応	距離・色・反射光・押下・赤外線3機能を実装。
ジャイロ	対応	角度・回転速度・比較・プログラム内の角度リセットを実装。実機検証待ち。
並行処理・センサイベント・メッセージ	対応	実装済み。
分岐・繰り返し・変数・基本演算	対応	小数を含め実装済み。
マイブロック	対応	数値の引数と最大8個の戻り値、別作品への書き出し・読み込みを実装。
本体画面の文字・数値・画像	対応	日本語固定文字・数値・画像10種を実装。
ランプ・音	対応。音声再生も可。	3色ランプ・周波数指定の音。音声ファイル未対応。
本体ボタン	対応	旧作品用の内部実装はあり、新規一覧から非表示。
ポートの値の表示・自動保存	対応	値の表示と、編集内容のブラウザ内自動保存。
データロギング	標準機能なし。旧EV3 Labにはあり。	本体CSV記録・PCへの取り出しを実装。
USB接続	OSによる。Windows/Macは対応、ChromeOS版などは非対応との公開資料。	WebHIDを実装。対象端末での実機検証を継続。

ジャイロのリセットは、Block Labでは現在の測定値をプログラム内の0度として記録します。センサ本体の再校正は行いません。接続パネルの角度表示は生値のため、この基準点を反映しません。詳しくは第7章を参照してください。

この取り組みで大切にすること

学校が保有するEV3を、老朽化したWindows PCに代わって児童のChromebookで活用し続けることが目的です。AIによるコード生成と、人による授業設計・操作性の検討・実機テストを組み合わせて開発しています。

機能の多さだけでなく、児童が迷わず操作し、考え、試し、工夫する過程を大切にします。今後は学年に合う表現・表示方法、「○ます進む・さがる」なども検討します。これらの学年別機能は現在未実装です。

参照資料

LEGO Education 動作環境： https://education.lego.com/en-gb/product-resources/mindstorms-ev3/downloads/system-requirements/

アフレル 基本比較（データロギング・USB等）： https://afrel.co.jp/technology-info/information-ev3/ev3-software/ev3-ev3-classroom/44135/

アフレル モーター比較： https://afrel.co.jp/technology-info/information-ev3/ev3-software/ev3-ev3-classroom/44140/

アフレル 並行処理・イベント： https://afrel.co.jp/technology-info/information-ev3/ev3-software/ev3-ev3-classroom/44145/

アフレル 表示・音： https://afrel.co.jp/technology-info/information-ev3/ev3-software/ev3-ev3-classroom/44143/

本アプリはLEGO公式ソフトではありません。保存形式.ev3lab・.ev3blocksは本アプリ専用で、Classroomや旧EV3 Labの作品ファイルとは相互読み込みできません。